Universidad de Caldas

Facultad de Ingenierías — Ingeniería de Sistemas y Computación

Análisis y Diseño de Algoritmos

Proyecto Semestral 2026-B

Reparto de presupuesto y selección de participantes

Subproyecto 2 — Análisis de complejidad y NP-dureza (solución base)

7 de noviembre de 2026

Integrantes

Juan Manuel Martínez Sánchez 46703
Juan David Reyes Ortiz 54587

Docente: Luz Enith Guerrero Mendieta

Índice

1. Introducción **3**

2. Preliminares **3**
2.1. Notación asintótica . . . 3
2.2. Modelo de costo . . . 3
2.3. Problemas de decisión, P, NP y reducciones . . . 3
2.4. El conjunto potencia 2^V . . . 4

3. Formulación matemática de los problemas **6**
3.1. Problema 1: reparto óptimo del presupuesto (Rep-Pres) . . . 6
3.2. Problema 2: selección de participantes (Sel-Sub) . . . 6
3.3. Versiones de decisión . . . 8
3.4. Qué problema es el "B" de la reducción . . . 8

4. Algoritmos y funciones de eficiencia **9**
4.1. Parte 1: algoritmo voraz con montículo . . . 9
4.1.1. Algoritmo . . . 9
4.1.2. Optimalidad (argumento de intercambio) . . . 9
4.1.3. Función de eficiencia por conteo de operaciones . . . 10
4.1.4. Pseudo-polinomialidad y presupuesto continuo . . . 11
4.2. Parte 2: algoritmo exacto (búsqueda exhaustiva) . . . 12
4.2.1. Recurrencia sin poda: $\Theta(2^N)$ . . . 12
4.2.2. Recurrencia con cupo k: $\binom{N}{k}$ evaluaciones . . . 13
4.2.3. Por qué esto no es polinomial . . . 13
4.3. Parte 2: algoritmo voraz de aproximación . . . 13
4.4. Resumen de funciones de eficiencia . . . 15

5. NP-dureza de la selección de participantes **16**
5.1. Paso 1: problema de partida, Cobertura Máxima . . . 16
5.2. Paso 2: construcción de la reducción φ . . . 16
5.2.1. La instancia construida es válida . . . 17
5.3. Paso 3: la transformación es polinomial . . . 17
5.4. Paso 4: la transformación conserva la respuesta . . . 18
5.5. Paso 5: conclusión . . . 18
5.6. Ejemplo de la reducción . . . 19

6. Conclusiones **20**

A. Código de los algoritmos **21**

1 Introducción

Una red de N nodos entrena de forma colaborativa un modelo durante varias rondas, sin compartir datos crudos. Cada actualización se protege con ruido calibrado: más ruido protege más, pero reduce la utilidad de la actualización. En cada ronda hay que tomar dos decisiones:

1. **Selección:** qué subconjunto de nodos participa.
2. **Reparto:** cuánto del presupuesto de ruido se le asigna a cada nodo seleccionado.

Este informe estudia la dificultad computacional de ambas decisiones. Se demuestra que:

- el **reparto del presupuesto** (Parte 1) se resuelve de forma exacta en tiempo polinomial con un algoritmo voraz, cuya función de eficiencia es $\Theta(N + B \log N)$ en el peor caso;
- la **selección de participantes** (Parte 2) es **NP-difícil**, mediante una reducción desde Max-Coverage, de modo que no existe un algoritmo exacto de tiempo polinomial a menos que P = NP; por ello se usa un algoritmo voraz de aproximación con garantía $(1 - 1/e)$ y costo $\Theta(N\,k)$ evaluaciones.

2 Preliminares

2.1 Notación asintótica

Sean $T, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. Se usan las definiciones formales (Cormen et al. 2009):

Definición 2.1 (Cota superior, O)**.** $T(n) \in O(g(n))$ si existen constantes $c > 0$ y $n_0 \in \mathbb{N}$ tales que $T(n) \leq c\,g(n)$ para todo $n \geq n_0$.

Definición 2.2 (Cota inferior, Ω)**.** $T(n) \in \Omega(g(n))$ si existen constantes $c > 0$ y $n_0 \in \mathbb{N}$ tales que $T(n) \geq c\,g(n)$ para todo $n \geq n_0$.

Definición 2.3 (Cota ajustada, Θ)**.** $T(n) \in \Theta(g(n))$ si $T(n) \in O(g(n))$ y $T(n) \in \Omega(g(n))$; es decir, existen $c_1, c_2 > 0$ y n_0 tales que $c_1 g(n) \leq T(n) \leq c_2 g(n)$ para todo $n \geq n_0$.

Observación 2.4 (Peor caso, mejor caso y cota del algoritmo)*.* Cuando el número de operaciones depende de los datos (no solo del tamaño), se distinguen $T_{\text{peor}}(n)$ y $T_{\text{mejor}}(n)$. En ese caso se reporta: $T(n) \in O(T_{\text{peor}})$, $T(n) \in \Omega(T_{\text{mejor}})$, y una cota Θ para cada caso por separado. Cuando el algoritmo realiza siempre las mismas operaciones para un tamaño dado, $T_{\text{peor}} = T_{\text{mejor}}$ y el algoritmo tiene una única cota Θ.

2.2 Modelo de costo

Se usa el modelo RAM: las operaciones aritméticas, comparaciones, asignaciones y accesos a arreglos cuestan tiempo constante. Para la Parte 2 la utilidad conjunta f se consulta como una función; el costo de una evaluación de f se denota τ (como en el enunciado) y se reporta por separado, de modo que el costo total se expresa como (número de evaluaciones) $\times\tau$.

2.3 Problemas de decisión, P, NP y reducciones

Las clases P y NP se definen sobre **problemas de decisión**, cuya respuesta es *sí* o *no*. Un problema de optimización ("maximizar f") se convierte en uno de decisión agregando un umbral

t ("¿existe una solución con valor $\geq t$?").

Definición 2.5 (Reducción polinomial)**.** Un problema A se *reduce polinomialmente* a un problema B, y se escribe $A \leq_p B$, si existe una función φ, calculable en tiempo polinomial, que transforma cada instancia I de A en una instancia $\varphi(I)$ de B tal que

$$I \text{ tiene respuesta } \textit{sí} \text{ en } A \iff \varphi(I) \text{ tiene respuesta } \textit{sí} \text{ en } B.$$

Definición 2.6 (NP-difícil y NP-completo)**.** Un problema B es *NP-difícil* si todo problema de NP se reduce polinomialmente a B. Es *NP-completo* si además $B \in \text{NP}$.

Teorema 2.7 (Herramienta de la reducción)**.** *Si A es NP-difícil y $A \leq_p B$, entonces B es NP-difícil.*

Demostración. Sea $X \in \text{NP}$. Como A es NP-difícil, existe φ_1 polinomial con $X \leq_p A$. Por hipótesis existe φ_2 polinomial con $A \leq_p B$. La composición $\varphi_2 \circ \varphi_1$ es polinomial (la composición de polinomios es un polinomio) y conserva la respuesta, porque cada transformación la conserva. Así $X \leq_p B$ para todo $X \in \text{NP}$. ∎

Observación 2.8 (Dirección de la reducción)*.* Para demostrar que *nuestro* problema B es difícil, la reducción va **desde** un problema A ya conocido como difícil **hacia** B. La interpretación es: si existiera un algoritmo polinomial para B, se podría resolver A en tiempo polinomial transformando sus instancias con φ. Reducir en la dirección contraria ($B \leq_p A$) solo mostraría que B *no es más difícil* que A, lo cual no aporta nada.

2.4 El conjunto potencia 2^V

Para un conjunto finito V, 2^V denota el *conjunto potencia*: el conjunto de **todos** los subconjuntos de V. Una función de la forma $f : 2^V \to \mathbb{R}_{\geq 0}$ recibe un *subconjunto* de V y devuelve un número.

Lema 2.9. *Si $|V| = N$, entonces $|2^V| = 2^N$ y* $\displaystyle\sum_{k=0}^{N} \binom{N}{k} = 2^N.$

Demostración. Para formar un subconjunto, cada uno de los N elementos tiene dos opciones independientes: pertenecer o no pertenecer. Por el principio multiplicativo hay $2 \cdot 2 \cdots 2 = 2^N$ subconjuntos. Agrupándolos por tamaño, hay exactamente $\binom{N}{k}$ subconjuntos de tamaño k, y al sumar sobre $k = 0, \ldots, N$ se cuenta cada subconjunto una sola vez. ∎

La Figura 1 muestra los subconjuntos de $V = \{1, 2, 3\}$ organizados en un árbol: a cada nodo solo se le agregan elementos mayores que su máximo, por lo que ningún subconjunto se repite. El nivel k del árbol contiene los $\binom{3}{k}$ subconjuntos de tamaño k, que son exactamente los candidatos del problema de selección con cupo k.

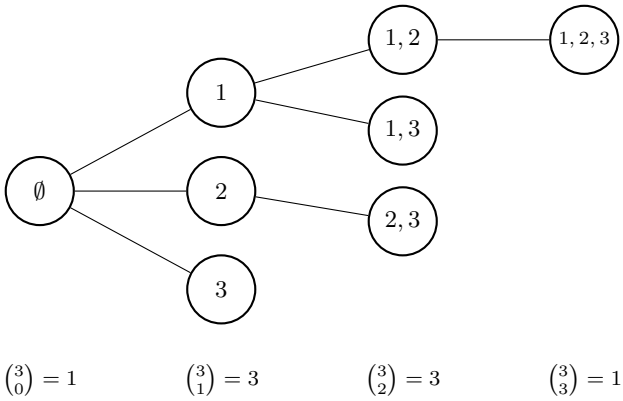


Figura 1: Árbol de los $2^3 = 8$ subconjuntos de $V = \{1, 2, 3\}$, agrupados por tamaño.

3 Formulación matemática de los problemas

3.1 Problema 1: reparto óptimo del presupuesto (Rep-Pres)

Definición 3.1 (Reparto óptimo del presupuesto)**. Entrada:** N nodos ya seleccionados para la ronda; para cada nodo $i \in \{1, \dots, N\}$, **su** función de utilidad $u_i : \mathbb{N} \to \mathbb{R}$, creciente y cóncava; y un presupuesto total $B \in \mathbb{N}$, medido en unidades enteras.
Salida: un reparto $(x_1, \dots, x_N) \in \mathbb{N}^N$, donde x_i es el número de unidades asignadas al nodo i.
Objetivo:

$$\max_{x_1,\dots,x_N} \sum_{i=1}^{N} u_i(x_i) \quad \text{sujeto a} \quad \sum_{i=1}^{N} x_i \le B, \qquad x_i \in \mathbb{N}.$$

Ganancia marginal. La ganancia de dar la unidad número $x+1$ al nodo i es

$$g_i(x) = u_i(x+1) - u_i(x), \qquad x = 0, 1, 2, \dots$$

Creciente significa $g_i(x) \ge 0$. **Cóncava** (en el caso discreto) significa que las ganancias no aumentan: $g_i(0) \ge g_i(1) \ge g_i(2) \ge \cdots$. Cada unidad adicional aporta igual o menos que la anterior.

Observación 3.2 ($\le B$ o $= B$). Como cada u_i es creciente, gastar una unidad sobrante nunca reduce la utilidad. Por lo tanto siempre existe un óptimo que usa exactamente B unidades, y el algoritmo de la Sección 4.1 las reparte todas.

Tamaño de la entrada y por qué es la medida adecuada. El problema crece con dos parámetros:

- N, el número de nodos: cada nodo agrega una función u_i a la entrada y un candidato más a cada decisión.
- B, el número de unidades de presupuesto: el voraz toma una decisión por unidad, así que B es el número de pasos.

Por eso la función de eficiencia es de dos variables, $T(N, B)$. Se supone que evaluar $u_i(x)$ cuesta tiempo constante, como en la utilidad implementada, $u_i(x) = w_i \ln(1+x)$, donde $w_i > 0$ mide la cantidad o calidad de los datos del nodo i. Con esta convención, la entrada se describe con N números w_i y el número B.

Observación 3.3 (Pseudo-polinomialidad). B se escribe con solo $\lceil \log_2(B+1) \rceil$ bits, pero el voraz hace B pasos. Su costo es polinomial en el *valor* de B y exponencial en la *longitud* de su representación; es decir, es **pseudo-polinomial**, el mismo fenómeno de la mochila 0/1. La Sección 4.1.4 lo discute y muestra cómo eliminar la dependencia en B.

3.2 Problema 2: selección de participantes (Sel-Sub)

Definición 3.4 (Selección de participantes)**. Entrada:**
Un conjunto de nodos candidatos $V = \{1, \dots, N\}$; una función de utilidad conjunta $f : 2^V \to \mathbb{R}_{\ge 0}$, que asigna a cada grupo de nodos $A \subseteq V$ su utilidad $f(A)$; y un cupo $k \in \mathbb{N}$ con $k < N$.

Salida:
Un grupo $A \subseteq V$ con $|A| = k$.

Objetivo:

$$\max_{A \subseteq V} \; f(A) \quad \text{sujeto a} \quad |A| \leq k.$$

Como f es monótona, siempre existe un óptimo con $|A| = k$, de modo que pedir $|A| \leq k$ o $|A| = k$ da el mismo valor óptimo.

Propiedades de f. Para $A \subseteq V$ y $j \notin A$ se define la ganancia marginal

$$\Delta_j(A) = f(A \cup \{j\}) - f(A),$$

es decir, lo que aporta el nodo j al grupo A. Se exige que f sea:

1. **Normalizada:** $f(\emptyset) = 0$.
2. **Monótona:** $A \subseteq B' \Rightarrow f(A) \leq f(B')$; equivalentemente, $\Delta_j(A) \geq 0$.
3. **Submodular:** para todo $A \subseteq B' \subseteq V$ y $j \notin B'$,

   $$\Delta_j(A) \;\geq\; \Delta_j(B').$$

   Agregar un nodo a un grupo pequeño aporta al menos lo que aporta agregarlo a un grupo más grande (rendimientos decrecientes). Se usa B', y no B, para no confundir este grupo con el presupuesto.
4. **Evaluable en tiempo polinomial:** f se da mediante una descripción compacta que permite calcular $f(A)$ en tiempo polinomial. Se denota τ el costo de una evaluación.

Observación 3.5 (Por qué la condición 4). El dominio 2^V tiene 2^N elementos (Lema 2.9). Si f se diera como una tabla con todos sus valores, la entrada tendría tamaño exponencial en N, y recorrerla parecería "lineal" en el tamaño de la entrada, ocultando la dificultad real. Por ello f se describe de forma compacta.

Utilidad implementada. En la implementación se usa la utilidad de **cobertura**: hay un universo U de categorías de datos, el nodo j tiene datos de las categorías $S_j \subseteq U$, y

$$f(A) = \Big| \bigcup_{j \in A} S_j \Big|.$$

Esta f cumple las condiciones 1–4 (Lema 5.4). Su descripción compacta son los conjuntos S_j.

Tamaño de la entrada y por qué es la medida adecuada.

- N, el número de candidatos: determina cuántos grupos existen (2^N, o $\binom{N}{k}$ con cupo) y cuántos candidatos revisa el voraz en cada paso.
- k, el cupo: determina cuántos pasos hace el voraz y el tamaño de los grupos que revisa el exacto.

El costo de leer la descripción de f queda dentro de τ. Por eso las funciones de eficiencia se expresan como $T(N, k)$ en número de evaluaciones de f, y el tiempo total como $T(N, k) \cdot \tau$. Para la cobertura, con $s = \max_j |S_j|$, la Sección 4.3 precisa τ: $O(s)$ en el voraz y $O(k\, s)$ en el exacto.

3.3 Versiones de decisión

Problema	Pregunta (sí/no)
Rep-Pres	Dados $u_1, \ldots, u_N$, B y un umbral t: ¿existe un reparto con $\sum_i x_i \leq B$ y $\sum_i u_i(x_i) \geq t$?
Sel-Sub	Dados V, f, k y un umbral t: ¿existe $A \subseteq V$ con $\lvert A \rvert \leq k$ y $f(A) \geq t$?

Nota: Introducimos el umbral t ya que sirve para convertir un problema de optimización en una pregunta de sí o no.

3.4 Qué problema es el "B" de la reducción

El proyecto completo (Red) combina las dos decisiones: elegir el grupo A con $|A| \leq k$ y repartir B entre sus nodos. Las dos partes tienen dificultades muy distintas:

- Rep-Pres está en P (Sección 4.1). Si se lograra reducir un problema NP-difícil hacia Rep-Pres, se tendría un algoritmo polinomial para un problema NP-difícil, lo que implicaría P = NP. Por lo tanto, **la reducción no puede ir hacia la Parte 1**.
- La dificultad del proyecto está en la **selección**. La reducción se construye hacia Sel-Sub (Sección 5).

Como Sel-Sub es un caso particular de Red (tomando utilidades que no dependen del presupuesto, por ejemplo con $B = 0$), cualquier algoritmo para Red resuelve Sel-Sub. En consecuencia, la NP-dureza de Sel-Sub se traslada al problema completo del proyecto.

4 Algoritmos y funciones de eficiencia

4.1 Parte 1: algoritmo voraz con montículo

4.1.1 Algoritmo

La idea es entregar las B unidades una por una, cada vez al nodo cuya *siguiente* unidad aporta más. Cada nodo i tiene **una** función de utilidad u_i (son N funciones en total, una por nodo, porque cada nodo tiene datos distintos). La ganancia de la unidad número $x+1$ del nodo i se obtiene con una resta,

$$g_i(x) = u_i(x+1) - u_i(x),$$

y se calcula **solo cuando se necesita**, no se construye una tabla de $N \times B$ ganancias. Un montículo de máximos guarda, para cada nodo, la ganancia de su próxima unidad.

Algoritmo 1: RepartoVoraz(N, u, B)

Input: N nodos; para cada nodo i, **su** función de utilidad u_i (cóncava y creciente); presupuesto $B \geq 1$
Output: unidades asignadas a cada nodo, $x_1, \ldots, x_N$, con $\sum_i x_i = B$

```
1 for i ← 1 to N do x_i ← 0   // el nodo i empieza sin presupuesto;   // para cada nodo
2 H ← lista vacía;
3 for i ← 1 to N do agregar (g_i(0), i) a H        // 1.ª unidad del nodo i, con su u_i;
4 Construir-Montículo-Max(H) ;                                          // O(N)
5 for b ← 1 to B do
6     (g, i) ← Extraer-Máximo(H) ;          // nodo con mayor ganancia, O(log N)
7     x_i ← x_i + 1 ;                                  // se le da una unidad
8     Insertar(H, (g_i(x_i), i)) ;        // solo la siguiente ganancia de i, O(log N)
9 return (x_1, ..., x_N);
```

Correspondencia con la implementación. Cada línea del pseudocódigo corresponde a una instrucción de la función `reparto_voraz`, que está en el archivo `code/algoritmos.py`:

Línea	Python
1	`x = [0] * N`
2–3	`H = [(-g(i, 0), i) for i in range(N)]`
4	`heapq.heapify(H)`
5	`for _ in range(B):`
6	`_, i = heapq.heappop(H)`
7	`x[i] += 1`
8	`heapq.heappush(H, (-g(i, x[i]), i))`

Como `heapq` es un montículo de *mínimos*, las ganancias se guardan con signo negativo: la mayor ganancia queda como el número más pequeño y sale primero.

4.1.2 Optimalidad (argumento de intercambio)

Teorema 4.1. *El reparto x^g producido por el Algoritmo 1 es óptimo.*

Demostración. **Paso 1: la utilidad se escribe como suma de ganancias.** Para cualquier

reparto x,

$$\sum_{i=1}^{N} u_i(x_i) \;=\; \sum_{i=1}^{N} u_i(0) \;+\; \sum_{i=1}^{N} \sum_{y=0}^{x_i-1} g_i(y),$$

por suma telescópica. El primer término no depende del reparto, así que maximizar la utilidad equivale a maximizar la suma de las B ganancias marginales usadas.

Paso 2: el voraz toma las B mayores ganancias. En cada iteración el montículo contiene la próxima ganancia de cada nodo, y por concavidad esa es la mayor ganancia aún disponible de ese nodo. Por lo tanto el voraz toma, en cada paso, la mayor ganancia disponible entre todas. **Sea λ la menor ganancia que tomó el voraz**. Entonces toda ganancia tomada es $\geq \lambda$, y toda ganancia no tomada es $\leq \lambda$.

Paso 3: intercambio. Sea x^* un reparto óptimo con $x^* \neq x^g$. Por la observación que sigue a la Definición 3.1, se puede tomar x^* con $\sum_i x_i^* = B$, igual que el voraz. Como ambos suman B, existen nodos i y j con $x_i^* > x_i^g$ y $x_j^* < x_j^g$. Se construye x' moviendo una unidad de i a j en x^*. El cambio en la utilidad es

$$\underbrace{g_j(x_j^*)}_{\text{unidad que el voraz sí tomó}} \;-\; \underbrace{g_i(x_i^* - 1)}_{\text{unidad que el voraz no tomó}} \;\geq\; \lambda - \lambda \;=\; 0,$$

porque $x_j^* < x_j^g$ implica que el voraz tomó la unidad x_j^* de j, y $x_i^* - 1 \geq x_i^g$ implica que no tomó la unidad $x_i^* - 1$ de i. Así x' también es óptimo y está más cerca de x^g (la distancia $\sum_i |x_i^* - x_i^g|$ baja en 2).

Paso 4: conclusión. Repitiendo el intercambio un número finito de veces se llega a x^g sin disminuir la utilidad. Por lo tanto x^g tiene utilidad igual a la del óptimo. ■

4.1.3 Función de eficiencia por conteo de operaciones

Sean $c_1, \ldots, c_8 > 0$ constantes del modelo RAM (se supone que evaluar u_i, y por lo tanto g_i, cuesta tiempo constante). Un montículo binario con N elementos tiene altura $h = \lfloor \log_2 N \rfloor$.

Línea	Operación	Costo por ejecución	Veces
1	inicializar x_i	c_1	N
2	crear la lista	c_2	1
3	calcular $g_i(0)$ y agregarla	c_3	N
4	construir montículo	$\leq c_4 N$ (CLRS, cap. 6)	1
6	extraer el máximo	entre c_6 y $c_6\, h$	B
7	incrementar x_i	c_7	B
8	calcular $g_i(x_i)$ e insertar	entre c_8 y $c_8\, h$	B

La línea 5 (control del ciclo) se absorbe en las constantes de las líneas 6–8.

Peor caso. Al extraer y al insertar, el elemento recorre toda la altura del montículo (h niveles):

$$T_{\text{peor}}(N,B) = c_1 N + c_2 + c_3 N + c_4 N + \sum_{b=1}^{B} \big(c_6 \lfloor \log_2 N \rfloor + c_7 + c_8 \lfloor \log_2 N \rfloor\big)$$

$$= (c_1 + c_3 + c_4)\, N + c_2 + c_7\, B + (c_6 + c_8)\, B \lfloor \log_2 N \rfloor.$$

Para $N \geq 2$ se cumple $\frac{1}{2} \log_2 N \leq \lfloor \log_2 N \rfloor \leq \log_2 N$, $c_2 \leq c_2 N$ y $B \leq B \log_2 N$, de modo que con $c_{\text{mín}} = \min_i c_i$ y $c_{\text{máx}} = \max_i c_i$:

$$\tfrac{c_{\text{mín}}}{2} \big(N + B \log_2 N\big) \;\leq\; T_{\text{peor}}(N,B) \;\leq\; 4\, c_{\text{máx}} \big(N + B \log_2 N\big).$$

$$T_{\text{peor}}(N,B) \quad \in \quad \Theta(N \;+\; B \log N)$$

Mejor caso. Si los elementos se detienen tras una comparación al extraer y al insertar (por ejemplo, cuando todas las ganancias son iguales):

$$T_{\text{mejor}}(N, B) = (c_1 + c_3 + c_4')\,N + c_2 + (c_6 + c_7 + c_8)\,B \;\in\; \Theta(N + B),$$

donde $0 < c_4' \le c_4$ es el costo por elemento de construir el montículo (la construcción siempre recorre los N elementos).

Cotas del algoritmo. Siguiendo la Observación 2.4:

$$T(N, B) \in O(N + B \log N), \qquad T(N, B) \in \Omega(N + B).$$

Cuando $B \ge N$ (caso usual, más unidades que nodos), $N + B \log N \in \Theta(B \log N)$, que coincide con la cota $O(B \log N)$. En todos los casos el algoritmo es polinomial en N y en el *valor* de B; la Sección 4.1.4 precisa qué significa esto.

4.1.4 Pseudo-polinomialidad y presupuesto continuo

Por qué la cota es pseudo-polinomial. El tamaño de la entrada en bits es del orden de N (los parámetros w_i) más $\lceil \log_2(B+1) \rceil$ (el número B). El voraz hace B iteraciones, y $B = 2^{\log_2 B}$: si se duplica el número de *dígitos* de B, el tiempo se eleva al cuadrado. Por eso $O(N + B \log N)$ es polinomial en el *valor* de B pero exponencial en la *longitud* de su representación, igual que la programación dinámica $O(nW)$ de la mochila 0/1. En la práctica es eficiente siempre que B sea un número moderado (en el proyecto, las unidades de presupuesto de una ronda).

Cómo quitar la dependencia en B (método del umbral). La prueba de optimalidad muestra que el voraz toma exactamente las ganancias mayores que un umbral λ. Para un λ dado, el número de unidades del nodo i con ganancia $\ge \lambda$ es

$$c_i(\lambda) = \big|\{x \ge 0 : g_i(x) \ge \lambda\}\big|,$$

y como g_i es no creciente, $c_i(\lambda)$ se obtiene con búsqueda binaria sobre $x \in \{0, \dots, B\}$ en $O(\log B)$ evaluaciones. Con una segunda búsqueda binaria sobre λ se encuentra el umbral para el que $\sum_i c_i(\lambda)$ alcanza B, y las unidades empatadas en λ se reparten hasta completar B. Si las ganancias son enteras y acotadas por G, el costo es $O(N \log B \log G)$, polinomial en el número de bits de la entrada. Esto confirma que Rep-Pres está en P aun cuando B es muy grande. Para los tamaños del proyecto se usa el voraz con montículo, que es más simple y es el que pide el enunciado.

Presupuesto continuo (nivelación, *water-filling*). Si $x_i \in \mathbb{R}_{\ge 0}$ y las u_i son derivables, cóncavas y crecientes, el problema es convexo y las condiciones de optimalidad (KKT) dicen que existe $\lambda \ge 0$ con

$$u_i'(x_i) = \lambda \;\text{ si } x_i > 0, \qquad u_i'(0) \le \lambda \;\text{ si } x_i = 0, \qquad \sum_i x_i = B.$$

Es la versión continua del umbral: todos los nodos que reciben presupuesto terminan con la *misma* ganancia marginal λ, como agua que se nivela. Para la utilidad implementada, $u_i(x) = w_i \ln(1+x)$, se tiene $u_i'(x) = w_i/(1+x)$ y por lo tanto

$$x_i(\lambda) = \max\!\Big(0,\; \frac{w_i}{\lambda} - 1\Big).$$

Como $\sum_i x_i(\lambda)$ es decreciente en λ, el nivel se encuentra ordenando los w_i y probando cuántos nodos quedan activos, en $O(N \log N)$, o con búsqueda binaria sobre λ en $O(N \log(1/\varepsilon))$ para precisión ε. En el caso continuo el costo ya no depende de B, porque no hay unidades que entregar una por una.

4.2 Parte 2: algoritmo exacto (búsqueda exhaustiva)

Como Sel-Sub es NP-difícil (Sección 5), no se conoce un algoritmo exacto polinomial. El algoritmo exacto natural recorre el árbol de decisiones "incluir / no incluir" cada nodo.

Nota: en esta entrega el algoritmo exacto se implementó por fuerza bruta; en la próxima entrega se reemplazará por una solución con programación dinámica. Como su costo es exponencial, solo es viable para pocos nodos; esta limitación es conocida y es la que se resolverá en la próxima entrega.

Algoritmo 2: SelecciónExacta(V, f, k)

Input: nodos $V = \{1, \ldots, N\}$; utilidad f; cupo k
Output: el mejor grupo A^* con $|A^*| = k$ y su utilidad $v^* = f(A^*)$

```
1  función Exacto(i, A):
2      if |A| = k then // caso 1: el cupo ya está lleno
3          return (f(A), A);
4      quedan ← N − i + 1 ;                          // nodos que faltan por decidir
5      faltan ← k − |A| ;                            // lugares libres en el cupo
6      if quedan < faltan then // caso 2: no alcanzan los nodos
7          return (−∞, ∅);
8      (v1, A1) ← Exacto(i + 1, A ∪ {i}) ;           // caso 3a: incluir el nodo i
9      (v2, A2) ← Exacto(i + 1, A) ;                 // caso 3b: no incluir el nodo i
10     if v1 ≥ v2 then
11         return (v1, A1);
12     else
13         return (v2, A2);
14 (v*, A*) ← Exacto(1, ∅) ;                         // llamada inicial
15 return (v*, A*);
```

4.2.1 Recurrencia sin poda: $\Theta(2^N)$

Si se elimina el cupo y se recorren todos los subconjuntos (la recursión llega siempre hasta $i > N$ y evalúa f en cada hoja), cada llamada con n nodos por decidir hace dos llamadas con $n-1$ y trabajo constante c:

$$T(n) = 2\,T(n-1) + c, \qquad T(0) = \tau.$$

Se resuelve por sustitución (desplegando):

$$\begin{aligned} T(n) &= 2T(n-1) + c \\ &= 2\big(2T(n-2) + c\big) + c = 2^2 T(n-2) + c(2+1) \\ &= 2^3 T(n-3) + c(2^2 + 2 + 1) \\ &\;\;\vdots \\ &= 2^j T(n-j) + c \sum_{l=0}^{j-1} 2^l = 2^j T(n-j) + c\,(2^j - 1). \end{aligned}$$

Con $j = n$: $T(n) = 2^n \tau + c\,(2^n - 1)$. Por lo tanto

$$T(N) \;=\; (\tau + c)\,2^N - c \;\;\in\;\; \Theta\big(2^N\big) \text{ (con } \tau \text{ constante), o } \Theta(2^N \tau) \text{ en general.}$$

4.2.2 Recurrencia con cupo k: $\binom{N}{k}$ evaluaciones

Sea $L(n, r)$ el número de evaluaciones de f (hojas) cuando quedan n nodos por decidir y faltan r por escoger. De las dos llamadas recursivas (líneas 8 y 9):

$$L(n, r) = L(n-1,\, r-1) + L(n-1,\, r), \qquad L(n, 0) = 1, \qquad L(n, r) = 0 \text{ si } r > n.$$

Es la recurrencia del triángulo de Pascal, cuya solución es $L(n, r) = \binom{n}{r}$ (por inducción, usando $\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}$). El algoritmo evalúa f exactamente $\binom{N}{k}$ veces, y como cada llamada interna conduce a alguna hoja y la profundidad es a lo sumo N, hay a lo sumo $N\binom{N}{k}$ llamadas internas. Entonces

$$T_{\text{exacto}}(N, k) \in \Omega\left(\binom{N}{k}\tau\right), \qquad T_{\text{exacto}}(N, k) \in O\left(\binom{N}{k}(N+\tau)\right),$$

y el número de evaluaciones es exactamente $\binom{N}{k}$, es decir $\Theta\left(\binom{N}{k}\right)$. El algoritmo hace siempre el mismo recorrido, así que el mejor y el peor caso coinciden.

4.2.3 Por qué esto no es polinomial

Lema 4.2. $\displaystyle\binom{N}{\lfloor N/2 \rfloor} \geq \frac{2^N}{N+1}.$

Demostración. Por el Lema 2.9, los $N+1$ números $\binom{N}{0}, \ldots, \binom{N}{N}$ suman 2^N. El mayor de ellos es $\binom{N}{\lfloor N/2 \rfloor}$, y el mayor de $N+1$ números es al menos su promedio, $2^N/(N+1)$. ∎

Así, para $k = \lfloor N/2 \rfloor$ el algoritmo exacto hace $\Omega(2^N/N)$ evaluaciones: crece de forma **exponencial**. Para k constante, $\binom{N}{k} \in \Theta(N^k)$, un polinomio cuyo grado crece con k; como k es parte de la entrada, el algoritmo no es polinomial en general. Este comportamiento es consistente con la NP-dureza: no se espera que exista un algoritmo exacto polinomial.

Observación 4.3 (Cota del algoritmo vs. cota del problema). Las cotas O, Ω y Θ anteriores son del *algoritmo exacto*. La NP-dureza no demuestra una cota inferior Ω exponencial para *todo* algoritmo posible; demuestra que un algoritmo polinomial implicaría P = NP.

4.3 Parte 2: algoritmo voraz de aproximación

Algoritmo 3: SelecciónVoraz(V, f, k)

Input: nodos $V = \{1, \ldots, N\}$, utilidad submodular monótona f, cupo k
Output: grupo A con $|A| = k$

```
1  A ← ∅;  f_A ← 0 ;                                  // grupo vacío, utilidad 0
2  for r ← 1 to k do                                  // una vuelta por cada lugar del cupo
3      mejor ← ninguno;  mejorDelta ← −∞;
4      for j ← 1 to N do
5          if j ∈ A then continuar                    // ya está en el grupo;
6          δ ← f(A ∪ {j}) − f_A ;                     // Δ_j(A): una evaluación de f
7          if δ > mejorDelta then
8              mejor ← j;  mejorDelta ← δ;
9      A ← A ∪ {mejor};  f_A ← f_A + mejorDelta;
10 return A;
```

Correspondencia con la implementación.
En la función `seleccion_voraz` del archivo `code/algoritmos.py` la utilidad es la de cobertura, $f(A) = |C(A)|$. En lugar de guardar f_A se guarda el conjunto `cubierto` $= C(A)$, y la ganancia de la línea 6 se calcula con el Lema 5.3: `delta = len(S[j] - cubierto)`, es decir, $\Delta_j(A) = |S_j \setminus C(A)|$. Al agregar el nodo se actualiza `cubierto |= S[mejor]`.

Conteo de evaluaciones. En la iteración i quedan $N - (i-1)$ candidatos, y se evalúa f una vez por candidato:

$$T_{\text{voraz}}(N,k) = \sum_{i=1}^{k} \big(N - i + 1\big) = kN - \frac{k(k-1)}{2}.$$

Como $k < N$, se tiene $\frac{k(k-1)}{2} < \frac{kN}{2}$, y por lo tanto

$$\frac{kN}{2} \;\le\; T_{\text{voraz}}(N,k) \;\le\; kN.$$

$$T_{\text{voraz}}(N,k) \;\in\; \Theta(N\,k) \text{ evaluaciones de } f\text{; tiempo total } \Theta(N\,k\,\tau).$$

El algoritmo realiza siempre las mismas iteraciones, así que el mejor y el peor caso coinciden: $O(Nk)$, $\Omega(Nk)$ y $\Theta(Nk)$.

Valor de τ para la utilidad implementada. El enunciado exige precisar τ. Con la utilidad de cobertura $f(A) = \big|\bigcup_{i\in A} S_i\big|$ y $s = \max_j |S_j|$:

- **Voraz.** Se mantiene el conjunto de categorías ya cubiertas, $C(A)$, en una tabla hash. Por el Lema 5.3, la ganancia es $\Delta_j(A) = |S_j \setminus C(A)|$, que se calcula recorriendo S_j: $\tau = O(s)$. Agregar el nodo elegido a $C(A)$ cuesta otro $O(s)$ por paso. En total,

  $$T_{\text{voraz}}(N,k) \in \Theta(N\,k) \text{ evaluaciones}, \qquad \text{tiempo } O(N\,k\,s).$$

- **Exacto.** Cada hoja calcula la unión de k conjuntos desde cero: $\tau = O(k\,s)$. En total, $\binom{N}{k}$ evaluaciones y tiempo $O\big(\binom{N}{k}(N + k\,s)\big)$.

Garantía de aproximación.

Teorema 4.4 (Nemhauser, Wolsey y Fisher 1978)**.** *Si f es normalizada, monótona y submodular, el grupo A_g del Algoritmo 3 cumple*

$$f(A_g) \;\ge\; \left(1 - \frac{1}{e}\right) f(A^*) \;\approx\; 0{,}632\, f(A^*),$$

donde A^* es un grupo óptimo de tamaño a lo sumo k.

Aplicación. Las hipótesis del teorema son exactamente las condiciones 1–3 de f en la formulación de Sel-Sub. Para verificarlo en los datos, en instancias pequeñas (donde el Algoritmo 2 es viable) se calcula la razón $f(A_g)/f(A^*)$ y se comprueba que es $\ge 1 - 1/e$.

La cota no se puede mejorar. Feige (1998) demostró que Max-Coverage no admite una aproximación mejor que $1-1/e$ en tiempo polinomial a menos que P = NP. Como Max-Coverage es un caso particular de Sel-Sub (Sección 5), el voraz alcanza la mejor garantía posible.

4.4 Resumen de funciones de eficiencia

Algoritmo	O	Ω	Θ
Reparto voraz (P1)	$N + B \log N$	$N + B$	peor: $N + B \log N$ mejor: $N + B$
Selección exacta con cupo (P2)	$\binom{N}{k}(N + \tau)$	$\binom{N}{k}\tau$	$\binom{N}{k}$ evaluaciones
Selección exacta sin cupo (P2)	$2^N \tau$	$2^N \tau$	$2^N \tau$
Selección voraz (P2)	$Nk\tau$	$Nk\tau$	Nk evaluaciones

5 NP-dureza de la selección de participantes

La demostración sigue cinco pasos:

1. Escoger un problema A conocido como NP-difícil: Max-Coverage.
2. Construir una transformación φ desde Max-Coverage hacia el problema B = Sel-Sub.
3. Demostrar que φ se calcula en tiempo polinomial.
4. Demostrar que φ conserva la respuesta: *sí* en Max-Coverage $\iff$ *sí* en Sel-Sub.
5. Concluir que Sel-Sub es NP-difícil.

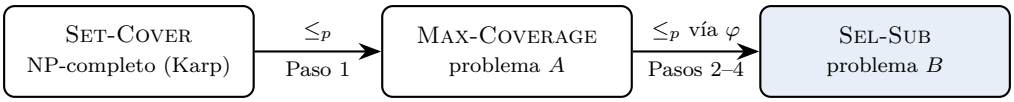


Figura 2: Cadena de reducciones. La dificultad se hereda en la dirección de las flechas.

5.1 Paso 1: problema de partida, Cobertura Máxima

Definición 5.1 (Max-Coverage)**.** **Entrada:** U, $\mathcal{S} = \{S_1, \ldots, S_m\}$, un entero k y un umbral t. **Pregunta:** ¿existen k conjuntos de $\mathcal{S}$ cuya unión tiene al menos t elementos?

Lema 5.2. *Max-Coverage es NP-completo.*

Por qué se escoge Max-Coverage. Su estructura coincide con la de nuestro problema: escoger k elementos (conjuntos o nodos) para maximizar algo en lo que las contribuciones se solapan. Un elemento cubierto por dos conjuntos cuenta una sola vez, igual que la información redundante entre dos nodos.

5.2 Paso 2: construcción de la reducción φ

Dada una instancia $I = (U, \mathcal{S} = \{S_1, \ldots, S_m\}, k, t)$ de Max-Coverage, se construye la instancia $\varphi(I) = (V, f, k', t')$ de Sel-Sub.

Caso trivial $k \geq m$. Sel-Sub exige $k < N$. Si $k \geq m$, en Max-Coverage se pueden tomar todos los conjuntos y, por monotonía, (con a lo sumo k conjuntos) la respuesta es *sí* exactamente cuando $\left|\bigcup_{i=1}^{m} S_i\right| \geq t$, lo cual se calcula en tiempo $O(\|I\|)$. En ese caso φ devuelve una instancia fija de Sel-Sub con la misma respuesta (por ejemplo, $V = \{1, 2\}$, $S_1 = \{a\}$, $S_2 = \emptyset$, $k' = 1$ y $t' = 1$ para *sí*, o $t' = 2$ para *no*). En adelante se supone $k < m$:

Componente	Valor construido	Razón
Nodos	$V = \{1, \ldots, m\}$, $N = m$	Cada conjunto S_i se convierte en el nodo i; escoger el nodo i equivale a escoger S_i.
Utilidad	$f(A) = \left\lvert \bigcup_{i \in A} S_i \right\rvert$	La utilidad de un grupo es cuántos elementos distintos cubre: exactamente lo que Max-Coverage cuenta.
Cupo	$k' = k$	Se escogen tantos nodos como conjuntos.
Umbral	$t' = t$	La utilidad mínima pedida es la cobertura mínima pedida.

Interpretación en el proyecto. U son categorías (o regiones) de datos y S_i son las categorías de las que el nodo i tiene datos. La utilidad de entrenar con un grupo es cuántas categorías distintas aporta; una categoría que ya aportó otro nodo no suma de nuevo.

Importante: V son los nodos (lo que se *escoge*) y U son los elementos (lo que se *cubre*).

Algoritmo 4: TRANSFORMAR$(U,\ S_1, \ldots, S_m,\ k,\ t)$

```
1 N ← m;
2 for i ← 1 to m do
3 |  nodo[i].conjunto ← copia de S_i;
4 return (N, nodo[1..N], k, t) ;          // la descripción de f son los conjuntos
```

Algoritmo 5: EVALUAR-$f(A)$

```
1 cubierto[1..|U|] ← FALSO;  cont ← 0;
2 foreach i ∈ A do
3 |  foreach e ∈ S_i do
4 |  |  if no cubierto[e] then
5 |  |  |  cubierto[e] ← VERDADERO;  cont ← cont + 1 ;  // cada elemento cuenta una
  |  |  |   vez
6 return cont;
```

5.2.1 La instancia construida es válida

Para que $\varphi(I)$ sea una instancia de SEL-SUB, f debe cumplir las condiciones de la formulación. Sea $C(A) = \bigcup_{i \in A} S_i$, de modo que $f(A) = |C(A)|$.

Lema 5.3 (Ganancia marginal de la cobertura)**.** *Para $A \subseteq V$ y $j \notin A$: $\Delta_j(A) = |S_j \setminus C(A)|$.*

Demostración. $C(A \cup \{j\}) = C(A) \cup S_j$ es la unión disjunta de $C(A)$ y $S_j \setminus C(A)$. Por lo tanto $f(A \cup \{j\}) = |C(A)| + |S_j \setminus C(A)|$, y al restar $f(A) = |C(A)|$ queda el resultado. En palabras: j aporta solo los elementos que todavía no estaban cubiertos. ∎

Lema 5.4. *La función $f(A) = |C(A)|$ es normalizada, monótona y submodular.*

Demostración. **Normalizada:** $f(\emptyset) = |\emptyset| = 0$.

Monótona: por el Lema 5.3, $\Delta_j(A) = |S_j \setminus C(A)| \geq 0$.

Submodular: sean $A \subseteq B' \subseteq V$ y $j \notin B'$.

1. $A \subseteq B'$ implica $C(A) \subseteq C(B')$, porque B' contiene todos los nodos de A.
2. Entonces $S_j \setminus C(B') \subseteq S_j \setminus C(A)$: al quitar un conjunto más grande queda menos.
3. Por lo tanto $|S_j \setminus C(B')| \leq |S_j \setminus C(A)|$.
4. Por el Lema 5.3, esto es $\Delta_j(B') \leq \Delta_j(A)$. ∎

5.3 Paso 3: la transformación es polinomial

Sea $\|I\| = m + |U| + \sum_{i=1}^{m} |S_i|$ el tamaño de la instancia de MAX-COVERAGE.

Costo de φ (Algoritmo 4). La línea 1 cuesta c_1; el ciclo copia cada S_i con costo $c_2|S_i| + c_3$:

$$T_\varphi(I) = c_1 + \sum_{i=1}^{m} \big(c_2|S_i| + c_3\big) = c_1 + c_3\, m + c_2 \sum_{i=1}^{m} |S_i| \ \in\ \Theta\Big(m + \sum_{i} |S_i|\Big) \subseteq O(\|I\|).$$

Costo de evaluar f (Algoritmo 5). La inicialización cuesta $\Theta(|U|)$ y el doble ciclo hace $\sum_{i \in A} |S_i|$ iteraciones de costo constante:

$$\tau(A) \in \Theta\Big(|U| + \sum_{i \in A} |S_i|\Big) \subseteq O(\|I\|).$$

Ambos costos son **lineales** en el tamaño de la entrada; en particular, polinomiales. Además, el tamaño de la salida es del mismo orden que el de la entrada, de modo que la instancia construida no crece de forma exponencial.

5.4 Paso 4: la transformación conserva la respuesta

Teorema 5.5. *I tiene respuesta* sí *en* Max-Coverage $\iff$ *$\varphi(I)$ tiene respuesta* sí *en* Sel-Sub.

Demostración. ($\Rightarrow$) Supóngase que existen índices $i_1, \ldots, i_k$ con $\left|S_{i_1} \cup \cdots \cup S_{i_k}\right| \geq t$. Sea $A = \{i_1, \ldots, i_k\}$. Entonces $|A| = k$ y

$$f(A) = \left|S_{i_1} \cup \cdots \cup S_{i_k}\right| \geq t,$$

así que $\varphi(I)$ tiene respuesta *sí*.

($\Leftarrow$) Supóngase que existe $A \subseteq V$ con $|A| \leq k$ y $f(A) \geq t$. Los conjuntos $\{S_i : i \in A\}$ son a lo sumo k y su unión tiene $f(A) \geq t$ elementos. Si son menos de k, se agregan conjuntos cualesquiera hasta completar k (es posible porque $k < m$); por la monotonía (Lema 5.4), la unión no disminuye y sigue teniendo al menos t elementos. Por lo tanto I tiene respuesta *sí*. ∎

Observación 5.6 (Por qué las dos direcciones). Con solo ($\Rightarrow$), una transformación que siempre produjera instancias con respuesta *sí* pasaría la prueba sin ser una reducción. La dirección ($\Leftarrow$) garantiza que φ no crea soluciones que no existían en la instancia original.

5.5 Paso 5: conclusión

Teorema 5.7. *Sel-Sub es NP-difícil; su versión de decisión es NP-completa.*

Demostración. Por el Lema 5.2, Max-Coverage es NP-difícil. Por los Pasos 2–4, φ es una reducción polinomial válida, es decir, Max-Coverage $\leq_p$ Sel-Sub. Por el Teorema 2.7, Sel-Sub es NP-difícil. Además, Sel-Sub está en NP: un grupo A sirve como certificado, y verificar $|A| \leq k$ y $f(A) \geq t$ toma tiempo polinomial porque f es evaluable en tiempo polinomial. ∎

Corolario 5.8. *La versión de optimización de Sel-Sub, y por la Sección 3.4 el problema completo Red, son NP-difíciles. Si existiera un algoritmo polinomial que encontrara un óptimo A^*, bastaría con comparar $f(A^*) \geq t$ para decidir Sel-Sub en tiempo polinomial, lo que implicaría* P = NP.

Consecuencia. A diferencia de la Parte 1, que se resuelve en $\Theta(N + B \log N)$, para la selección de participantes no se espera un algoritmo exacto polinomial: el exacto conocido hace $\Theta\big(\binom{N}{k}\big)$ evaluaciones, que es exponencial para $k \approx N/2$ (Lema 4.2). Por eso se usa el voraz de la Sección 4.3, con costo $\Theta(Nk)$ y garantía $(1 - 1/e)$.

5.6 Ejemplo de la reducción

Instancia con respuesta *sí*. $U = \{1,2,3,4,5,6\}$, $S_1 = \{1,2,3\}$, $S_2 = \{3,4\}$, $S_3 = \{4,5,6\}$, $S_4 = \{1,4\}$, $k = 2$, $t = 6$. Al aplicar φ: $V = \{1,2,3,4\}$, $f(A) = |C(A)|$, $k = 2$, $t = 6$.

A	$C(A)$	$f(A)$	¿$f(A) \geq 6$?
$\{1,2\}$	$\{1,2,3,4\}$	4	no
$\{1,3\}$	$\{1,2,3,4,5,6\}$	**6**	**sí**
$\{1,4\}$	$\{1,2,3,4\}$	4	no
$\{2,3\}$	$\{3,4,5,6\}$	4	no
$\{2,4\}$	$\{1,3,4\}$	3	no
$\{3,4\}$	$\{1,4,5,6\}$	4	no

Ambos problemas responden *sí*, con el testigo $A = \{1,3\}$ (S_1, S_3).

Submodularidad en números (nodo 2, $S_2 = \{3,4\}$).

A	$C(A)$	$S_2 \setminus C(A)$	$\Delta_2(A)$
$\emptyset$	$\emptyset$	$\{3,4\}$	2
$\{1\}$	$\{1,2,3\}$	$\{4\}$	1
$\{1,3\}$	$\{1,\ldots,6\}$	$\emptyset$	0

Instancia con respuesta *no*. Si se cambia $S_3 = \{4,5\}$, el elemento 6 no aparece en ningún conjunto, así que ninguna elección cubre los 6 elementos: Max-Coverage responde *no*, y en la instancia construida ningún par alcanza $f(A) = 6$, así que Sel-Sub también responde *no*.

6 Conclusiones

- El **reparto del presupuesto** entre nodos con utilidades cóncavas se resuelve de forma exacta con un algoritmo voraz, cuya optimalidad se demuestra por intercambio. Su función de eficiencia es $\Theta(N + B \log N)$ en el peor caso y $\Theta(N + B)$ en el mejor caso. Es polinomial en N y en el valor de B, es decir, pseudo-polinomial; el método del umbral, y la nivelación en el caso continuo, eliminan la dependencia en B.
- La **selección de participantes** con utilidad submodular es **NP-difícil**. Se demostró con la reducción Max-Coverage $\leq_p$ Sel-Sub: cada conjunto se convierte en un nodo y la utilidad es el tamaño de la unión; la transformación es lineal y conserva la respuesta en ambas direcciones.
- El algoritmo exacto de la selección realiza $\Theta\left(\binom{N}{k}\right)$ evaluaciones de f, con $\tau = O(k\,s)$ cada una; sin cupo, su recurrencia $T(n) = 2T(n-1) + c$ da $\Theta(2^N)$. Para $k = \lfloor N/2 \rfloor$ el costo es $\Omega(2^N/N)$, exponencial.
- El algoritmo voraz de selección cuesta $\Theta(N\,k)$ evaluaciones, con $\tau = O(s)$ cada una, y garantiza al menos $(1 - 1/e) \approx 63\,\%$ del óptimo (Nemhauser, Wolsey y Fisher 1978); esta garantía es la mejor alcanzable en tiempo polinomial a menos que P = NP (Feige 1998).
- Como la selección es un caso particular del problema completo del proyecto, la dificultad del proyecto se concentra en elegir los participantes, no en repartir el presupuesto.

Trabajo pendiente. Este informe cubre la solución base de las dos partes. Las variantes obligatorias (costos de participación, repetición en T rondas con fallas y nodos relacionados entre sí) se desarrollarán en la siguiente entrega. Para la variante con costos, la cota del voraz cambia y debe citarse en su versión para cobertura con presupuesto (Khuller, Moss y Naor 1999).

A Código de los algoritmos

El código completo, incluido el generador de instancias y las mediciones (`code/experimentos.py`), acompaña este informe. Para regenerar las tablas y gráficas de los experimentos basta ejecutar `python code/experimentos.py` desde la carpeta del informe.

```
"""
Algoritmos del Subproyecto 2: reparto de presupuesto y selección de participantes.

Parte 1 (reparto):    reparto_voraz      -> exacto, O(N + B log N)
Parte 2 (selección):  seleccion_exacta   -> exacto, C(N,k) evaluaciones de f
                      seleccion_voraz    -> aproximado (1 - 1/e), N*k evaluaciones de f

La utilidad de la Parte 2 es la de cobertura: f(A) = |unión de S[j] con j en A|.
"""
import heapq
import math


# ============================================================================
# PARTE 1: reparto voraz por ganancia marginal (Algoritmo 1 del informe)
# ============================================================================
def reparto_voraz(u, N, B):
    """u(i, x) = utilidad del nodo i si recibe x unidades (cóncava y creciente).
    Devuelve x[0..N-1] con sum(x) = B."""
    g = lambda i, x: u(i, x + 1) - u(i, x)        # ganancia marginal g_i(x)
    x = [0] * N                                   # línea 1
    H = [(-g(i, 0), i) for i in range(N)]         # líneas 2-3 (signo - => montículo de máximos)
    heapq.heapify(H)                              # línea 4, O(N)
    for _ in range(B):                            # línea 5
        _, i = heapq.heappop(H)                   # línea 6, O(log N)
        x[i] += 1                                 # línea 7
        heapq.heappush(H, (-g(i, x[i]), i))       # línea 8, O(log N)
    return x


def utilidad_log(w):
    """u_i(x) = w_i * ln(1 + x): creciente y cóncava, evaluable en O(1)."""
    return lambda i, x: w[i] * math.log1p(x)


def utilidad_total(u, x):
    return sum(u(i, xi) for i, xi in enumerate(x))


# ============================================================================
# PARTE 2: utilidad de cobertura
# ============================================================================
def f_cobertura(S, A):
    """f(A) = |unión de S[j], j en A|. Costo tau = O(sum_{j en A} |S[j]|)."""
    cubierto = set()
    for j in A:
        cubierto |= S[j]
    return len(cubierto)


# ============================================================================
# PARTE 2: selección exacta por fuerza bruta (Algoritmo 2 del informe)
# ============================================================================
def seleccion_exacta(S, k):
    """Recorre el árbol incluir / no incluir. Devuelve (v*, A*) y cuántas
    veces evaluó f (debe ser C(N, k))."""
    N = len(S)
    evaluaciones = [0]

    def exacto(i, A):
```

```
61          if len(A) == k:                                     # caso 1: cupo lleno (
    hoja)
62              evaluaciones[0] += 1
63              return f_cobertura(S, A), list(A)
64          quedan, faltan = N - i, k - len(A)                  # i va de 0 a N-1 en
    Python
65          if quedan < faltan:                                 # caso 2: poda
66              return -math.inf, []
67          A.append(i)
68          v1, A1 = exacto(i + 1, A)                           # caso 3a: incluir i
69          A.pop()
70          v2, A2 = exacto(i + 1, A)                           # caso 3b: no incluir i
71          return (v1, A1) if v1 >= v2 else (v2, A2)
72
73      v, A = exacto(0, [])
74      return v, A, evaluaciones[0]
75
76
77  # ======================================================================
78  # PARTE 2: selección voraz (Algoritmo 3 del informe)
79  # ======================================================================
80  def seleccion_voraz(S, k):
81      """Devuelve (f(A), A, evaluaciones). Cada evaluación de la ganancia
82      marginal cuesta tau = O(|S[j]|) gracias al conjunto 'cubierto'."""
83      N = len(S)
84      A, en_A, cubierto = [], [False] * N, set()
85      evaluaciones = 0
86      for _ in range(k):                                      # una vuelta por lugar
    del cupo
87          mejor, mejor_delta = -1, -1
88          for j in range(N):
89              if not en_A[j]:                                 # los que ya están en el
    grupo se saltan
90                  delta = len(S[j] - cubierto)                # Delta_j(A) = |S_j \ C(A
    )|
91                  evaluaciones += 1
92                  if delta > mejor_delta:
93                      mejor, mejor_delta = j, delta
94          A.append(mejor)
95          en_A[mejor] = True
96          cubierto |= S[mejor]
97      return len(cubierto), A, evaluaciones
98
99
100 if __name__ == "__main__":
101     # Ejemplo del informe (Parte 1)
102     u = utilidad_log([10, 8, 7])
103     print("Reparto voraz:", reparto_voraz(u, N=3, B=5))
104     # Ejemplo del informe (Parte 2)
105     S = [{1, 2, 3}, {3, 4}, {4, 5, 6}, {1, 4}]
106     print("Exacto:", seleccion_exacta(S, 2))
107     print("Voraz :", seleccion_voraz(S, 2))
108     # Instancia donde el voraz no es óptimo
109     S = [{1, 2, 3, 4}, {1, 2, 5}, {3, 4, 6}]
110     print("Exacto:", seleccion_exacta(S, 2), " Voraz:", seleccion_voraz(S, 2))
```

Referencias

Cormen, Thomas H. et al. (2009). *Introduction to Algorithms*. 3.ª ed. Cambridge, MA: MIT Press.

Feige, Uriel (1998). «A Threshold of $\ln n$ for Approximating Set Cover». En: *Journal of the ACM* 45.4, págs. 634-652.

Khuller, Samir, Anna Moss y Joseph Naor (1999). «The Budgeted Maximum Coverage Problem». En: *Information Processing Letters* 70.1, págs. 39-45.

Nemhauser, George L., Laurence A. Wolsey y Marshall L. Fisher (1978). «An Analysis of Approximations for Maximizing Submodular Set Functions—I». En: *Mathematical Programming* 14.1, págs. 265-294.